Technical Documentation

Architecture, data model, subsystems, security and operations for the Motion Graphics Archive Management System

System	MG Archive Bot
Document version	1.1
Status	Released for review
Date of issue	26 September 2026
Intended audience	Engineering, technical reviewers and system administrators
Document owner	Motion Graphics Ministry, Team Lead Group
Prepared by	Motion Graphics Archive Development Team
Classification	Internal use only
Software release	1.0.0

Contents

1 About this document . . . 1

1.1 Purpose . . . 1

1.2 Scope . . . 1

1.3 Intended readership . . . 1

1.4 Related documents . . . 1

1.5 Terminology . . . 1

1.6 Revision history . . . 2

2 System overview . . . 3

2.1 The problem . . . 3

2.2 The solution . . . 3

2.3 Capability summary . . . 3

2.4 Design principles . . . 4

2.5 Out of scope for release 1.0.0 . . . 4

3 Architecture . . . 5

3.1 Runtime topology . . . 5

3.2 Technology stack . . . 5

3.3 Package structure . . . 5

3.4 Lifecycle of an update . . . 6

3.5 Trust boundaries . . . 7

4 Data model . . . 8

4.1 Storage engine . . . 8

4.2 Entities . . . 8

4.3 The project record . . . 9

4.4 Enumerations . . . 9

4.5 Schema evolution . . . 9

5 Google Drive integration . . . 11

5.1 Authentication modes . . . 11

5.2 Permission model . . . 11

5.3 Client design . . . 11

5.4 Archive folder structure . . . 12

5.5 Provisioning . . . 12

5.6 Revocation and restoration . . . 12

5.7 Startup verification . . . 13

6 Archive lifecycle and validation . . . 14

6.1 Declarations . . . 14
6.2 Definition of a satisfied leaf . . . 14
6.3 Required leaves . . . 14
6.4 The scan algorithm . . . 14
6.5 Error semantics . . . 15
6.6 State machine . . . 15
6.7 Triggers and scheduling . . . 16
6.8 Audit trail . . . 16

7 Preview subsystem . . . 17

7.1 Objectives and constraints . . . 17
7.2 Source selection . . . 17
7.3 Change detection . . . 17
7.4 Worker and transcoding . . . 17
7.5 Delivery . . . 18

8 File listing . . . 19

8.1 Purpose . . . 19
8.2 Reading the tree . . . 19
8.3 Rendering . . . 19
8.4 Entry points . . . 19

9 Discovery and ranking . . . 21

9.1 Query normalisation . . . 21
9.2 Matching . . . 21
9.3 Ranking . . . 21
9.4 Result presentation . . . 21

10 Project index sheet . . . 22

10.1 Purpose . . . 22
10.2 Layout . . . 22
10.3 Synchronisation . . . 22
10.4 Resilience . . . 22

11 Telegram interface design . . . 23

11.1 Two modes . . . 23
11.2 Command surface . . . 23

11.2.1 Per-role command menus . . . 23
11.3 The creation wizard . . . 24
11.4 Callback routing . . . 24
11.5 Free text steps . . . 25
11.6 Message construction . . . 25
11.7 The live status message . . . 25
11.8 Group identity and renaming . . . 26
11.9 Chat migration . . . 26

12 Security . . . 27
12.1 Attack surface . . . 27
12.2 Authentication . . . 27
12.3 Authorisation . . . 27
12.4 The Lights role . . . 28
12.5 Group authorisation . . . 28
12.6 Visibility of registrations . . . 29
12.7 Abuse controls . . . 29
12.8 Input handling . . . 29
12.9 Secrets . . . 30
12.10 Logging and audit . . . 30

13 Concurrency, reliability and correctness . . . 31
13.1 Concurrency rules . . . 31
13.2 Failure behaviour . . . 31

14 Configuration reference . . . 33

15 Deployment and operations . . . 35
15.1 Requirements . . . 35
15.2 Installation . . . 35
15.3 Operator tools . . . 35
15.4 Startup sequence . . . 36
15.5 Scheduled jobs . . . 36
15.6 Backup and recovery . . . 36
15.7 Upgrade procedure . . . 37
15.8 Runbook . . . 37
15.9 Performance and capacity . . . 38

16 Quality assurance . . . 39

16.1 Approach . . . 39
16.2 Suite composition . . . 39
16.3 Running the suite . . . 39
16.4 Emphasis and gaps . . . 39

17 Constraints, risks and roadmap . . . 40
17.1 Known constraints . . . 40
17.2 Risk register . . . 40
17.3 Candidate future work . . . 41

Appendix A: Folder key reference . . . 42

Appendix B: Glossary of status labels . . . 43

1 About this document

1.1 Purpose

This document is the authoritative technical description of the **MG Archive Bot**, the Telegram based Motion Graphics Archive Management System that operates on top of the ministry's Google Drive archive. It records the architecture, the data model, the behaviour of every subsystem, the security posture, the operational procedures and the quality assurance approach in enough detail for an engineer who has never seen the codebase to maintain, extend, deploy and audit it.

1.2 Scope

The document covers release **1.0.0** of the application: the Telegram interface, the archive lifecycle and validation engine, the preview generation pipeline, the discovery and ranking engine, the Google Drive and Google Sheets integrations, the persistence layer, the security controls and the runtime operations.

It does not cover the ministry's editorial standards for motion graphics content, the specification of the ProRes and Hap delivery formats themselves, or the administration of Google Workspace. End user procedures are published separately in the *User Manual*.

1.3 Intended readership

Reader	What this document provides
Engineers maintaining or extending the system	Module map, data model, algorithms, invariants and concurrency rules
Technical reviewers and approvers	Design rationale, security model, risk register and known constraints
System administrators and operators	Configuration reference, deployment procedure, scheduled jobs and runbook
Auditors	Authentication and authorisation model, abuse controls, data handling and retention behaviour

1.4 Related documents

Reference	Location	Content
Requirements specification	`docs/REQUIREMENTS.md`	The originating business requirements
Design notes	`docs/DESIGN.md`	Working design record maintained alongside the code
User Manual	`docs/USER_MANUAL.pdf`	Task oriented instructions for designers, Team Leads and the Super Admin
Configuration template	`.env.example`	Annotated list of every environment variable

1.5 Terminology

Term	Definition
Archive	The complete folder tree on Google Drive for one production, plus its metadata record in the database
Project	The database record that represents one archive, its declarations, metadata, assignments and status
Collection	A folder directly under the archive root that groups several related sub-projects
Declaration	A Team Lead's statement that a project will deliver a given category of asset
Leaf	A folder in the archive tree that designers upload files into and that validation inspects
MG Group	A Telegram group chat that has been authorised by the bot to receive archive notifications
Provisioning token	A single use secret of the form `MG-XXXX-XXXX` that authorises one Telegram group
Preview	A small MP4 rendition of a ProRes master, stored on Drive and delivered as a link
Scan	One validation pass over a project's required leaves on Google Drive
Actor	The authenticated, active user record behind an incoming Telegram update

1.6 Revision history

Version	Date	Author	Summary
1.0	18 September 2026	Development Team	First released edition covering the complete 1.0.0 feature set
1.1	26 September 2026	Development Team	Adds the Lights role, the file listing subsystem, the live status message, the group name synchronisation, the registration notice and the password reset tool

2 System overview

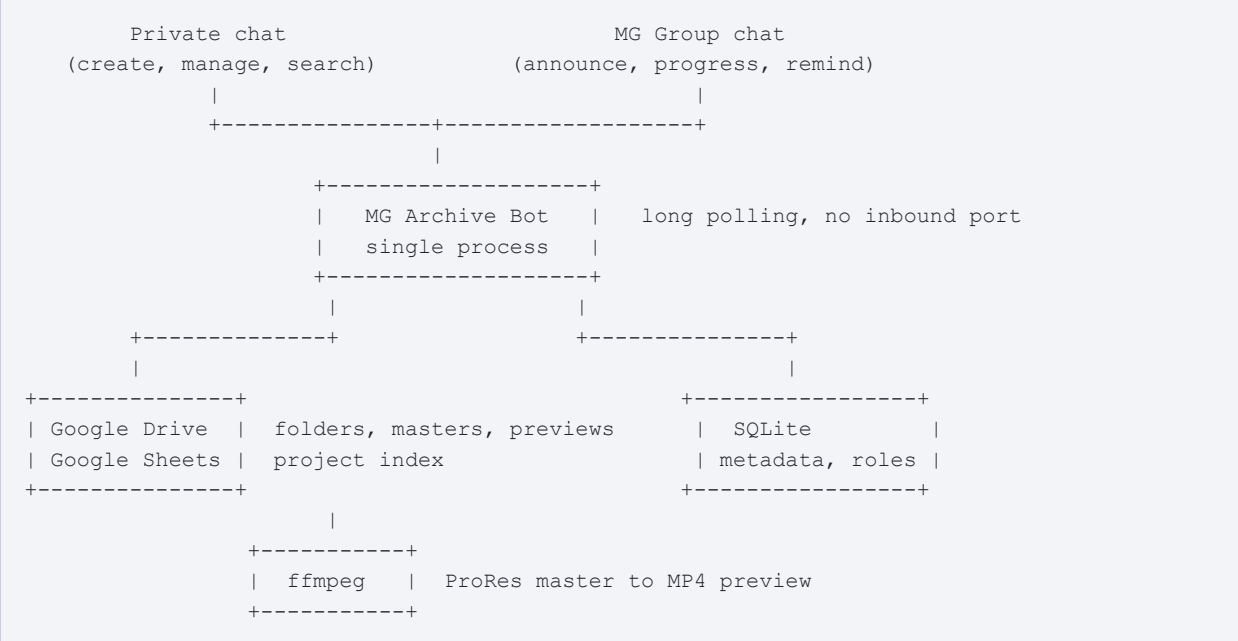
2.1 The problem

Before this system existed, every archive was assembled by hand. A Team Lead created the Google Drive folder tree, copied each link into a formatted message, posted the request into the Motion Graphics group chat, checked Drive manually to see what had arrived, chased the designers who were late, and transcribed the result into a tracking spreadsheet. The work was repetitive, easy to get wrong, and it scaled linearly with the number of productions.

Discovery was equally manual. Assets were stored correctly but could only be found by someone who remembered where they were, which made reuse of existing motion graphics rare and unpredictable.

2.2 The solution

The system is a single long running Python process that presents a Telegram bot to the ministry and treats Google Drive as the system of record for file content. It automates the four expensive activities: provisioning, tracking, chasing and discovery.



2.3 Capability summary

Capability	Behaviour
Archive provisioning	A guided wizard collects the location, name, declarations, target group, metadata and designer assignments, then creates the complete folder tree on Drive and posts the announcement
Progress tracking	Scheduled and on demand scans read Google Drive directly and move the project through its lifecycle states without any human checklist
Reminders	A daily job posts the outstanding items into the MG Group and mentions the responsible designers; Team Leads can also trigger a reminder on demand

Capability	Behaviour
Previews	ProRes masters in the declared delivery folders are transcoded to compact MP4 files stored on Drive and handed out as streaming links
Discovery	A private search command performs conjunctive matching across name, tags and metadata, ranked by match quality
Project index	A Google Sheet in the archive root holds one row per project and is kept in step automatically
Access control	Password based registration, four fixed roles, revocation, group authorisation by token and brute force protection

2.4 Design principles

The implementation is governed by seven principles. They are stated here because most of the detailed decisions later in this document follow from them.

1. **Google Drive is the source of truth.** Completion is never asserted by a person ticking a box. It is derived from what actually exists in Drive at the moment of the scan.
2. **Uncertainty is not progress.** If a folder cannot be listed, the scan records an error for that leaf and changes nothing: no state transition, no notification, no reminder. A failed check must never be mistaken for an empty folder.
3. **No inbound network surface.** The process uses Telegram long polling and makes only outbound HTTPS calls. Nothing listens on a port.
4. **Least privilege on Drive.** The bot's Google identity needs access to one folder. It never modifies sharing settings, and it trashes rather than permanently deletes.
5. **Separation of the two chat modes.** Management and discovery happen in private chat; the group chat receives announcements, progress and reminders only. This keeps the shared channel usable.
6. **Every state changing action is idempotent under retry.** Double taps, overlapping scans and restarts must not produce duplicate folder trees, duplicate transitions or duplicate transcodes.
7. **The database never blocks the event loop.** Every mutation is committed before the next network call, so no write transaction is ever held across an `await`.

2.5 Out of scope for release 1.0.0

The bot does not generate Hap or Hap Alpha renditions, it does not upload video content into Telegram, it does not manage Google Drive permissions, and it does not provide PNG previews for Contin Lyrics. These exclusions are deliberate and are discussed in section 17.

3 Architecture

3.1 Runtime topology

The application is one operating system process. Inside it, three cooperating activities share a single asyncio event loop.

Activity	Implementation	Responsibility
Update processing	`python-telegram-bot` application with concurrent updates enabled	Commands, button presses, membership changes
Scheduled jobs	The library's job queue, backed by APScheduler	Drive scans, daily reminders, nightly index rebuild, group name refresh, stale chat eviction
Preview worker	A single background asyncio task consuming a queue	Serialised download, transcode and upload of previews

Blocking work never runs on the event loop. Google API calls and `ffmpeg` invocations are dispatched to a thread with `asyncio.to_thread`, and the Drive client keeps a thread local service object because the underlying HTTP transport is not thread safe.

3.2 Technology stack

Concern	Choice	Rationale
Language	Python 3.11 or newer	Mature, first class client libraries for both Telegram and Google
Telegram	`python-telegram-bot` 22.8 with the job queue extra	Async application, inline keyboards, callback routing and scheduling in one dependency
Database	SQLite through SQLAlchemy 2.0	Zero administration for a single process workload; the connection URL can be repointed at PostgreSQL without code changes
Drive and Sheets	Google API Python client, Drive v3 and Sheets v4	Official, supports Shared Drives and resumable transfers
Credentials	Service account, or an OAuth user token	A service account suits Google Workspace and Shared Drives; OAuth covers a plain consumer account
Transcoding	`ffmpeg` with libx264	The de facto standard, available on every target platform
Configuration	`pydantic-settings` reading a `.env` file	Typed and validated at startup rather than at first use
Testing	`pytest` with `pytest-asyncio`	Deterministic async tests against in memory doubles

3.3 Package structure

```
mg_archive_bot/
  __main__.py          process entry point, logging, startup checks
  config.py            typed settings loaded from the environment
  constants.py         enumerations, folder tree specification, labels
  db.py                engine, session factory, schema creation and upgrade
  models.py            SQLAlchemy ORM models
  security.py          scrypt password hashing, provisioning token generation
  util.py              escaping, timezone handling, term normalisation, formatting
  services/            domain logic, independent of Telegram
    drive.py           Drive client protocol, Google implementation, in memory double
    sheets.py          Sheets client protocol, Google implementation, in memory double
    users.py           registration, roles, revocation, login lockout
    groups.py          MG groups, provisioning tokens, chat migration
    collections.py     collection folders under the archive root
    projects.py        creation, provisioning, metadata, assignments, transitions
    validation.py      the scan algorithm, reports and status transitions
    previews.py        preview planning, transcoding, upload and bookkeeping
    listing.py         folder by folder reading of a project's Drive folder
    search.py          normalisation, conjunctive matching, ranking
    tracking.py        the project index sheet layout and synchronisation
    notifications.py   message builders for private replies and group posts
  bot/                 the Telegram adapter
    app.py             application assembly, handler and job registration
    access.py          authorisation decorators, actor resolution, prompt state
    actions.py         operations shared by handlers and jobs
    keyboards.py       inline keyboard construction
    jobs.py            scheduled jobs and the preview worker
    ratelimit.py       sliding window limiter and circuit breaker
    handlers/          one module per interaction area
  tools/
    drive_check.py     diagnostic for Drive access and sharing
    google_oauth.py    one time OAuth token bootstrap
    reset_password.py  set the access password and clear login lockouts
```

The dependency direction is strict: `bot` depends on `services`, and `services` never imports from `bot`. Domain logic is therefore testable without a Telegram application, and the same functions are reused by interactive handlers and by scheduled jobs.

3.4 Lifecycle of an update

1. The library polls Telegram and receives an update. Only messages, callback queries and the bot's own chat membership changes are requested, which keeps the polling payload small.
2. The update is dispatched to the first matching handler. Command handlers are registered by name; button presses are routed by a regular expression over the callback payload namespace.
3. The `require` decorator resolves the actor from the Telegram user id, enforces the chat scope, the active status, the minimum role and, in a group, the group's authorisation. It also cancels any pending free text step so that a later message cannot be misread as an answer to an earlier prompt.
4. The handler opens a database session, performs its work, commits, and only then makes its outward calls to Telegram or Google.
5. Long running work is delegated: Drive and ffmpeg calls to worker threads, preview jobs to the queue, index sheet updates to a fire and forget task guarded by a lock.

3.5 Trust boundaries

Boundary	Direction	Control
Telegram to application	Inbound updates	Every command and every button press is authorised server side; crafted callback payloads are rejected because authorisation is never derived from the payload
Application to Telegram	Outbound messages	All interpolated user text is HTML escaped; link previews are disabled
Application to Google	Outbound API calls	Scoped credentials, one archive root folder, trash instead of permanent deletion
Application to `ffmpeg`	Local subprocess	Arguments are constructed as a list, never a shell string; Drive file names are never used as local path components
Operator to application	Configuration	Secrets live only in the environment file and the secrets directory, both excluded from version control

4 Data model

4.1 Storage engine

The default store is a single SQLite file. On every connection the engine enables foreign key enforcement, switches the journal to write ahead logging, sets synchronous mode to normal and applies a five second busy timeout. Write ahead logging allows the reads performed by scans to proceed while a handler commits.

An in memory database is explicitly rejected at startup, because a shared in memory connection would break session isolation and silently defeat the transactional guarantees the handlers rely upon.

All timestamps are stored as naive UTC values and converted to the configured local zone only for display.

4.2 Entities

```
          users --< assignments >-- projects >-- collections
            |                          |  |  |
    login_attempts                     |  |  +--< project_folders
                                       |  +-----< preview_assets
          settings                     |  +-----< validation_runs
                                       +--------< project_tags >-- tags

   mg_groups --< provisioning_tokens
   unauthorised_chats
```

Table	Key	Purpose
`users`	`telegram_id`	Identity, role and access status of every registered person
`login_attempts`	`telegram_id`	Failure counter and lock expiry for password brute force protection
`settings`	`key`	Key and value store for the access password hash and the index sheet identifiers
`mg_groups`	`chat_id`	Authorised Telegram groups, including who authorised them and how they were revoked
`provisioning_tokens`	`id`, unique `token`	Single use group authorisation secrets with an expiry
`unauthorised_chats`	`chat_id`	Chats the bot was added to without authorisation, so the eviction job can leave them
`collections`	`id`, unique `name`	Grouping folders under the archive root, created lazily on first use
`projects`	`id`	The central record: declarations, metadata, Drive identifiers, status and audit timestamps
`project_folders`	`id`	One row per provisioned folder, mapping a stable key to a Drive identifier and link
`assignments`	`id`	Which user is responsible for which asset category on which project
`tags`, `project_tags`	`id`	Normalised lower case keywords and their many to many association
`preview_assets`	`id`	One row per source master: fingerprint, generated preview, size and status

Table	Key	Purpose
validation_runs	id	Append only audit trail of every completed scan and its full report

4.3 The project record

The projects table carries six groups of columns.

Group	Columns	Notes
Identity	id, name, collection_id, status, mg_group_chat_id	The display name is Collection / Project when the project belongs to a collection
Declarations	has_timeline, has_contin_videos, has_contin_lyrics, has_psd	These four flags drive folder creation, validation and preview sourcing
Metadata	collection, event, ministry, style, colours, year, creator, asset_types, description	Free text used by discovery; free text fields are capped at 200 characters and the description at 2000
Drive linkage	drive_root_id, drive_link	Populated when the tree is provisioned
Live status message	status_message_id, status_message_hash	Identify the single progress message the bot maintains in the MG Group, and the digest of its current text so an unchanged message is never rewritten
Audit	created_by, created_at, activated_at, last_validated_at, last_complete, last_reminder_at, verified_by, verified_at, cancelled_by, cancelled_at, created_at, updated_at	Supports the lifecycle, the reminder interval and the index sheet

Project names are validated to between 2 and 100 characters, are rejected if they contain angle brackets, and must be unique, case insensitively, within their collection or within the top level.

4.4 Enumerations

Enumeration	Values
Role	SUPER_ADMIN, TEAM_LEAD, DESIGNER, LIGHTS
UserStatus	ACTIVE, REVOKED
GroupStatus	ACTIVE, REVOKED
ProjectStatus	DRAFT, ACTIVE, INCOMPLETE, READY_FOR_VERIFICATION, ARCHIVED, CANCELLED
AssetCategory	ALL, WORKING_FILE, TIMELINE, CONTIN_VIDEOS, CONTIN_LYRICS, PSD
PreviewStatus	PENDING, READY, FAILED

4.5 Schema evolution

Table creation is idempotent, but creating tables alone does not help an existing database when a newer release adds a column. At startup the application inspects every mapped table and issues an ALTER TABLE

`... ADD COLUMN` for each column that is missing. The routine is deliberately additive and refuses to add a column that is a primary key or that is both non nullable and without a default, because such a change cannot be applied safely without a data migration. This keeps ordinary upgrades to a restart while still failing loudly when a genuine migration is required.

5 Google Drive integration

5.1 Authentication modes

Mode	Credential	When to use	Consequence
`service_account`	JSON key file	Google Workspace with a Shared Drive. Recommended.	Files created by the bot are owned by the Shared Drive, not by a personal account
`oauth`	User token produced by the bootstrap tool	A plain consumer Google account with no Workspace	Files are owned by that user; the consent screen must be internal or published, otherwise tokens expire after seven days
`fake`	None	Demonstrations, evaluation and the automated test suite	Every flow works in Telegram against an in memory Drive; nothing is written to Google

5.2 Permission model

The bot's Google identity requires content manager rights on exactly one folder, the archive root. Sharing that single folder is sufficient; membership of the whole Shared Drive is not required. Listings are issued with the `allDrives` corpora so that the identity may reach the folder either as a Shared Drive member or through a direct share.

A Shared Drive is strongly preferred. In a personal My Drive, every folder the bot creates would be owned by the robot account and would count against its storage quota, and preview uploads would eventually fail. The application detects this configuration at startup and logs an explicit warning.

The bot never changes sharing settings. Designers see the archive because they are members of the Shared Drive or because the root folder was shared with them by an administrator.

5.3 Client design

The Drive client is defined as a protocol with nine operations, which allows the in memory double used by the tests and by demonstration mode to be substituted wherever the real client is expected.

Aspect	Implementation
Blocking model	Every method blocks; callers dispatch them with `asyncio.to_thread`
Thread safety	The API service object is stored in thread local storage because the HTTP transport is not thread safe
Retries	Requests execute with five automatic retries for transient failures
Transfers	Downloads use 32 MB chunks, uploads use resumable 16 MB chunks
Error contract	Every client, transport and authentication failure is translated into a single `DriveError` type, so callers have one exception to handle. Credential refresh failures carry a remediation hint
Query safety	Folder names and identifiers are escaped before being interpolated into a Drive query string
Deletion	Deletion is implemented as a move to trash, because content managers on a Shared Drive may trash but not permanently delete. Trashed items are recoverable for 30 days

5.4 Archive folder structure

Every project receives the same tree. Folder names match the requirements exactly and are individually overridable through configuration. Note that `Hap/Hap Alpha` is a single folder name; Drive permits the slash character inside a name.

```
<Project Name>/
  Working File/
    Fonts/                  required always
    AE/                     required always
    PSD/                    created and required only when PSD is declared
  Final Render/
    Timeline/
      ProRes 4444/          required when Timeline is declared
      Hap/Hap Alpha/        required when Timeline is declared
    Contin Videos/
      ProRes 4444/          required when Contin Videos is declared
      Hap/Hap Alpha/        required when Contin Videos is declared
    Contin Lyrics/
      PNG/                  required when Contin Lyrics is declared
  _Previews/                managed by the bot, not part of the delivery specification
```

Each folder is recorded in `project_folders` against a stable key. The key, not the display name, is what the validation and preview subsystems refer to, so renaming a folder through configuration does not invalidate existing records. Appendix A lists every key.

5.5 Provisioning

Provisioning runs inside a per project lock and proceeds as follows.

1. Resolve the parent. For a top level project this is the archive root. For a project inside a collection, the collection folder is created under the root if it does not exist, or an existing folder with the same name is reused.
2. Choose a unique folder name. If a folder with the project name already exists under the parent, a numeric suffix is appended, up to a bounded number of attempts.
3. Create the project root, then create each specified folder in parent before child order. The optional PSD folder is created only when declared.
4. Record every created folder with its Drive identifier and public link, set the project's Drive root and link, move the project from `DRAFT` to `ACTIVE`, and stamp the activation time.

The entire creation step runs in a worker thread. If any part fails the transaction is rolled back and the draft remains intact, so the Team Lead can correct the problem and retry rather than starting again.

When a declaration is enabled after creation, a separate routine creates only the folders that are missing, reusing any folder that already exists with the expected name.

5.6 Revocation and restoration

Revoking a project moves its Drive folder to the trash **before** the database status changes, so a Drive failure leaves the project untouched. Preview rows are removed, because their files were trashed together with the parent folder. Restoration takes the folder back out of the trash and returns the project to the `ACTIVE` state. Both operations are reflected in the index sheet and announced in the MG Group.

5.7 Startup verification

Before the bot starts polling it fetches the archive root. If the folder is not visible, the process exits with a message naming the sharing step that is missing. If the root resolves to a My Drive location while the service account mode is in use, the ownership and quota warning described above is logged. A listing of the root is then performed, which proves that the resolved corpora and drive identifiers actually work. A standalone diagnostic, `python -m mg_archive_bot.tools.drive_check`, performs the same checks and prints the exact permission to request when something is missing.

6 Archive lifecycle and validation

6.1 Declarations

Folder existence does not imply that an asset is required. During creation the Team Lead declares which categories the project will deliver: Timeline, Contin Videos, Contin Lyrics and PSD. The full delivery tree is still created, with the single exception of the PSD folder, but only declared categories are validated. An empty Timeline folder on a project that declared no Timeline assets is therefore not a defect.

6.2 Definition of a satisfied leaf

A leaf is satisfied when it contains at least one qualifying file. The search is recursive to three levels, so a designer may upload a sub folder rather than loose files without being reported as incomplete.

A file does not qualify if its name begins with a full stop, if it is one of the known operating system metadata files (`desktop.ini`, `thumbs.db`, `.DS_Store`), or if it is zero bytes. This prevents the artefacts that file synchronisation clients scatter through a folder tree from being counted as a delivery.

6.3 Required leaves

Leaf key	Path	Required when
`fonts`	Working File / Fonts	Always
`ae`	Working File / AE	Always
`psd`	Working File / PSD	PSD declared
`timeline_prores`	Final Render / Timeline / ProRes 4444	Timeline declared
`timeline_hap`	Final Render / Timeline / Hap/Hap Alpha	Timeline declared
`contin_prores`	Final Render / Contin Videos / ProRes 4444	Contin Videos declared
`contin_hap`	Final Render / Contin Videos / Hap/Hap Alpha	Contin Videos declared
`lyrics_png`	Final Render / Contin Lyrics / PNG	Contin Lyrics declared

6.4 The scan algorithm

```
for each leaf in the folder tree specification:
    required = (leaf is unconditional) or (the project declared its category)
    if not required:
        record "not declared" and continue
    if the leaf has no recorded Drive id:
        record "folder not provisioned", count 0, not satisfied
        continue
    try:
        files = breadth first listing under the leaf, maximum depth 3,
                excluding folders and excluding junk
    except DriveError:
        record the leaf as ERRORED and continue
    if the leaf is a preview source:
        remember the video files found for the preview planner
    record file count and satisfied = (count > 0)

complete = every required leaf is satisfied
```

The scan is executed in a worker thread from a plain snapshot of the project's declaration flags and folder identifiers. No ORM object crosses the thread boundary.

6.5 Error semantics

If any required leaf could not be listed, the report is marked as having errors and three things follow. The report's completeness is forced to false, no `validation_runs` row is written, and no status transition is applied. The user interface shows a warning that Google Drive could not be checked rather than a red cross, and the reminder job skips the project entirely. A transient Drive outage or an expired credential can therefore never cause a project to be reported as incomplete, and can never trigger a reminder that blames a designer for a file that is actually present.

6.6 State machine

```
   DRAFT --(wizard confirmed, folders created)--> ACTIVE
      |
      +--(wizard cancelled)--> deleted

   ACTIVE / INCOMPLETE / READY --(scan: incomplete)--> INCOMPLETE
   ACTIVE / INCOMPLETE         --(scan: complete)----> READY_FOR_VERIFICATION
   READY_FOR_VERIFICATION      --(Team Lead verify)--> ARCHIVED
   ARCHIVED                    --(Team Lead reopen)--> ACTIVE
   ACTIVE / INCOMPLETE / READY --(Team Lead revoke)--> CANCELLED
   CANCELLED                   --(Team Lead restore)-> ACTIVE
```

State	Meaning	Scanned	In search	In index sheet
`DRAFT`	The wizard is still in progress; no folders exist yet	No	No	No
`ACTIVE`	Provisioned and open for uploads	Yes	Yes	Yes
`INCOMPLETE`	At least one declared asset is missing	Yes	Yes	Yes

State	Meaning	Scanned	In search	In index sheet
`READY_FOR_VERIFICATION`	Everything declared is present, awaiting human sign off	Yes	Yes	Yes
`ARCHIVED`	Verified and closed by a Team Lead	No	Yes	Yes
`CANCELLED`	Revoked; the Drive folder is in the trash	No	No	No, the row is removed

Two invariants are worth stating explicitly. Verification is offered only when the most recent scan was complete, and a fresh scan is always run immediately before the confirmation is presented, so a Team Lead cannot archive a project on the strength of a stale result. Archived projects are not rescanned automatically, which means a later change on Drive does not silently reopen a closed archive.

6.7 Triggers and scheduling

Trigger	Cadence	Scope
Scheduled scan	Every `SCAN_INTERVAL_MINUTES`, default 30, first run two minutes after start	All projects in `ACTIVE`, `INCOMPLETE` or `READY_FOR_VERIFICATION`
Project menu check	On demand	One project
Group status command	On demand, reusing a result younger than the cooldown	Up to six open projects linked to that group
Reminder job	Daily at `REMINDER_HOUR` in the configured timezone	Projects that are open, linked to a group and not reminded within `REMINDER_MIN_GAP_HOURS`
Verification	On demand, immediately before the confirmation prompt	One project

6.8 Audit trail

Every scan that completed without Drive errors appends a row to `validation_runs` holding the timestamp, the completeness flag and the full report as JSON. The most recent row is what the project details view and the group progress message render, which means the interface can always show the last known state without issuing a fresh Drive call.

7 Preview subsystem

7.1 Objectives and constraints

Designers already produce ProRes 4444 masters for archival and Hap renditions for playback. Neither is suitable for review on a phone: the masters are frequently several gigabytes. The subsystem produces a compact H.264 rendition of every ProRes master in the declared delivery folders and makes it available as a link that streams in the Google Drive player.

Three constraints shaped the design. Telegram imposes a 50 MB limit on bot uploads, so video is never uploaded to Telegram. Transcoding is expensive in disk, bandwidth and CPU, so it is serialised and never retried blindly. Drive file names may contain slashes and relative path segments, so they are never used as local path components.

7.2 Source selection

Previews are generated from `timeline_prores` when Timeline is declared, and from `contin_prores` when Contin Videos is declared. A file qualifies if its MIME type begins with `video/` or its extension is one of `.mov`, `.mxf`, `.mp4`, `.m4v`, `.avi`, `.mkv` or `.prores`. Contin Lyrics produces no MP4 preview.

7.3 Change detection

Each source file is identified by its Drive identifier and a content fingerprint, which is the MD5 checksum when Drive supplies one and the size and modification time otherwise. The planner compares the stored fingerprint with the current one and decides as follows.

Stored row	Fingerprint	Action
None	n/a	Create a `PENDING` row and enqueue
`READY`	Unchanged	Nothing to do
`READY`	Changed	Reset to `PENDING`, enqueue, and delete the superseded MP4 after the new one is produced
`FAILED`	Unchanged	Skip during scheduled scans; retried only when a Team Lead requests generation explicitly
`PENDING`	Unchanged	Enqueue again; enqueue is idempotent per source file and fingerprint
Any	Source no longer present	Delete the row and remove the orphaned MP4 from Drive

Not retrying failures automatically is a deliberate decision. A failure usually means a multi gigabyte download, so an automatic retry every thirty minutes would consume bandwidth indefinitely for a file that will keep failing. Unattended failures are instead reported by direct message to the project's creator.

7.4 Worker and transcoding

The worker is a single asyncio task consuming a queue, so at most one transcode runs at a time. For each job it verifies that the stored fingerprint still matches, then performs the blocking sequence in a thread.

1. Verify free disk space. The job requires 1.3 times the source size plus 200 MB in the working directory and fails early with a clear message if that is not available.
2. Download the master into a per job working directory under a fixed, safe file name.
3. Transcode.
4. Delete the downloaded master before uploading, so peak disk usage stays close to the size of the source rather than the size of the source plus the output.
5. Upload the MP4 into the project's `_Previews/` folder, named after the source file's stem.
6. Record the Drive identifier, link and size, and set the row to `READY`.

The working directory is removed whether the job succeeded or failed. Leftovers from an interrupted run are purged at startup, and the affected rows are re-planned by the next scan.

Parameter	Value	Purpose
Video codec	`libx264`, preset `medium`	Universally playable in the Drive player
Quality	Constant rate factor, `PREVIEW_CRF`, default 26	Small files at review quality
Scaling	Width limited to `PREVIEW_MAX_WIDTH`, default 1280, rounded to an even number, height proportional	Avoids odd dimensions that the encoder rejects
Pixel format	`yuv420p`	Required for wide device compatibility
Container	MP4 with `+faststart`	The player can begin streaming before the whole file has transferred
Audio	AAC at 128 kbit/s	Adequate for review
Timeout	Four hours per file	Bounds a pathological job

The command line is assembled as an argument list, never as a shell string. Running processes are tracked so that they can be terminated promptly when the bot shuts down, which prevents an orphaned encoder from holding the machine after the service stops.

7.5 Delivery

Previews are delivered as links only. A preview request replies privately with a button that opens the MP4 in the Google Drive player. The multi gigabyte master is never opened, nothing is cached in Telegram, and there is no file size ceiling to manage. If a preview is requested before it is ready, the reply states the current status instead.

8 File listing

8.1 Purpose

Progress reporting answers whether a required folder holds at least one file. It deliberately says nothing about *which* files those are. The listing subsystem answers the second question: it reads a project's Drive folder in full and renders it, folder by folder, as a set of Telegram messages in which every folder and every file is a link.

It is used for three things: a Team Lead confirming that what arrived is what was asked for, a designer checking their own upload without opening Drive, and anyone inspecting a past archive found through search.

8.2 Reading the tree

`services/listing.py` builds an in-memory tree of `FolderEntry` and `FileEntry` records. The read is blocking and runs in a worker thread.

Aspect	Behaviour
Depth	Six levels below the project folder, which allows three levels of designer sub-folders beneath a leaf such as `Working File / Fonts`. Anything deeper is reported as not shown rather than silently omitted
Ordering	Folders the bot created keep their archive order, so the tree reads in the same sequence as the specification; every other folder is alphabetical, as are files
Exclusions	The same junk filter the validator uses, so operating system metadata and zero byte files are left out of both counts and listings
Counts	Each folder shows the total number of files beneath it, including its sub-folders, or is marked empty
Failures	A folder that cannot be listed is marked individually and the rest of the tree is still rendered. One unreadable folder never costs the whole listing

8.3 Rendering

`render_listing` turns the tree into Telegram HTML. The header carries the project name, the total file count and the summed size, and links to the project folder. Sizes are formatted per file, and every name is a link: a folder opens in Drive, a file opens in Drive's viewer.

Telegram caps a message at 4096 characters after parsing, so output is split at 3500 raw characters and the parts are numbered. Splitting happens between lines, so a file entry is never cut in half.

8.4 Entry points

Entry point	Who	Scope
Project menu, **Files**	Team Lead	One project
Search result card, **Files**	Everyone except Lights	One project, including archived ones
`/files` in an MG Group	Everyone in that group except Lights	The group's open archives, up to six, most recent first

A listing is reused for `STATUS_COOLDOWN_SECONDS`, the same window that governs the group status command, and read from Drive again after that. The window is short enough that the listing still describes the archive as it is rather than as the last scan saw it, and long enough that several people asking at once in a group produce one read rather than one each.

9 Discovery and ranking

9.1 Query normalisation

A query is split on commas, or on whitespace when it contains no comma. Each term is trimmed, collapsed internally, lower cased, stripped of a leading hash character and de-duplicated. Empty terms are discarded and the list is capped at twenty terms.

9.2 Matching

Matching is conjunctive. Every term must match at least one field, otherwise the project is not a result. Searchable fields are the project name, tags, collection, event, ministry, style, colours, year, creator, asset types and description. Cancelled and draft projects are never returned.

9.3 Ranking

Each term contributes the score of the best tier in which it matched. The tiers implement the ranking order from the requirements.

Rank	Tier	Weight
1	Exact tag match	100,000,000
2	Project name contains the term	1,000,000
3	Event or collection contains the term	10,000
4	Other metadata, or a partial tag match	100
5	Description contains the term	1

The weights are separated by more than the twenty term cap. The consequence is important and is the reason for the specific values: a single exact tag match always outranks any number of lower tier matches, so the published ranking order is strict rather than approximate.

Ties are broken in three stages: a project whose whole name equals the query comes first, then the more recent year, then alphabetical order. Results are paginated at `SEARCH_PAGE_SIZE`, which defaults to five, with a button to fetch the next page.

Scoring runs in the application over the candidate set rather than in SQL. For an archive of this size the cost is negligible and the ranking logic stays directly testable. Section 17 records the point at which an index backed approach would become worthwhile.

9.4 Result presentation

Each result is a card showing the display name, the tags, the number of available previews and the current status, with four actions: play a preview, open the archive folder on Drive, expand the full details including metadata, folder links and the latest validation summary, or list the files. A reader holding the Lights role is offered the preview alone.

10 Project index sheet

10.1 Purpose

The index is a human readable, long lived view of the archive: a Google Sheet named *MG Archive Index* held in the archive root, with one row for every project ever created. The database remains the source of truth; the sheet is a projection maintained for people who want to filter, sort or read the archive outside Telegram.

10.2 Layout

The sheet holds twenty five columns in a single tab with a frozen, bold header row and a basic filter.

Group	Columns
Identity	ID, Collection, Project, Status, Year
Metadata	Event, Ministry, Style, Colours, Tags, Asset types
Declarations	Timeline, Contin Videos, Contin Lyrics, PSD
People and provenance	Assigned, Created by, Created, Archived, Verified by, MG Group
Operational	Previews, Last checked, Drive link, Description

10.3 Synchronisation

Rows are upserted by project identifier. A synchronisation is scheduled after creation, after any metadata, declaration, assignment or group change, after verification or reopening, and after any status transition produced by a scan. Revocation deletes the row so that the rows below move up; restoration writes it back.

Synchronisations are dispatched as fire and forget tasks serialised by a single lock, so an interactive handler never waits for the Sheets API. Outstanding tasks are awaited during shutdown.

10.4 Resilience

The index is explicitly not allowed to break the main flow. Every failure is caught, logged, and reported once by direct message to the Super Admin, after which the bot continues normally. A nightly job at 03:30 local time rewrites the whole sheet from the database, which repairs anything a live synchronisation missed, and `/sheet rebuild` performs the same repair on demand.

If the Google Sheets API has not been enabled in the Cloud project, the error is detected specifically and reported with the exact remediation step rather than as a generic failure.

Capacity is not a practical concern: Google Sheets permits ten million cells, which at twenty five columns is roughly four hundred thousand projects.

11 Telegram interface design

11.1 Two modes

Private chat is the management and discovery interface. Group chat is a notification surface that accepts three commands. Any management or discovery command sent in a group is refused with a short message directing the user to a private chat, which is what keeps the shared channel readable.

11.2 Command surface

Private chat command	Minimum role	Function
`/start`	None	Register with the access password, or show the menu
`/search <terms>`	Designer	Search the archive; plain text is also treated as a search
`/whoami`	Designer	Show the caller's role, status and Telegram identifier
`/cancel`	Designer	Abort the step in progress
`/help`	Designer	List the commands available to the caller
`/newproject`	Team Lead	Run the archive creation wizard
`/projects`, `/project <id>`	Team Lead	List and manage archives
`/creategroup`	Team Lead	Issue a provisioning token for a new MG Group
`/sheet`, `/sheet rebuild`	Team Lead	Link to the project index, or rewrite it
`/users`	Super Admin	Manage users, roles and access
`/groups`	Super Admin	Manage MG Groups
`/setpassword`	Super Admin	Rotate the access password

Group command	Minimum role	Function
`/status`	Designer	Bring this group's live progress messages to the bottom of the chat with fresh figures
`/files`	Designer	List, folder by folder, what has been uploaded for this group's open archives. Refused for Lights
`/remind`	Team Lead	Post reminders for missing uploads
`/activate <token>`	Team Lead	Authorise this group with a provisioning token
`/help`	None	List the group commands

11.2.1 Per-role command menus

Telegram stores a command list per *scope* and serves the most specific scope matching a chat. It knows nothing of this application's roles, but a private chat's id is the user's id, so pushing a list to that chat's scope is how a role becomes a menu.

Two broad scopes are registered at startup: a viewer list for all private chats, which is what a stranger and a newly registered Designer see, and the group list for all group chats. On top of those, every active user receives a list chosen from their role.

Role	Private menu
Lights, Designer	`/start`, `/search`, `/whoami`, `/cancel`, `/help`
Team Lead	those plus `/newproject`, `/projects`, `/creategroup`, `/sheet`
Super Admin	those plus `/users`, `/groups`, `/setpassword`

Lights and Designers share a menu because the Lights restriction applies to what a search result offers and to `/files` in a group, not to which commands exist in a private chat.

Because Telegram caches the list rather than asking for it, a menu only changes when something pushes it. The push therefore happens at registration, when the Super Admin changes somebody's role, and for every active user at startup, which is what applies a change made while the bot was stopped. A failed push is logged and ignored: the consequence is a stale menu, not a loss of function, and the next start repairs it.

None of this is an access control. Authorisation is enforced by the `require` decorator on every handler, and a command typed by hand is refused exactly as before. The menu simply stops advertising commands the person cannot use.

11.3 The creation wizard

`/newproject` gathers a project in seven stages: where it lives, the name, the asset declarations, the MG Group, optional metadata, the designer assignment, and a summary that must be confirmed before anything is created on Drive.

The draft row exists from the name stage onward, so every later answer is persisted rather than held in memory and a restart mid wizard loses only the step in progress. A draft that is abandoned without `/cancel` survives until the same Team Lead successfully names their next one, at which point creating that draft deletes their earlier ones. Nothing is provisioned on Drive until the final confirmation, so an abandoned draft leaves no folders behind.

The MG Group stage is always presented. An earlier release linked the project silently when exactly one group was authorised. That shortcut was removed: a Team Lead running several productions, each with its own chat, could not see where the announcement was about to go, and an announcement posted into the wrong chat cannot be recalled. The stage therefore lists every active group, oldest first, followed by an option to defer, and repeats in the same message how to authorise a group that is missing. Where the group is left unset, the summary says so explicitly rather than showing a blank, because a project with no group is announced nowhere and tracked silently.

11.4 Callback routing

Button payloads use a compact namespace so a single regular expression can route each family to its handler. The payload carries the target and the action, never the caller's rights: authorisation is always re-evaluated server side, so a crafted payload gains nothing.

Namespace	Meaning	Example
`nw:`	Creation wizard steps	`nw:decl:TIMELINE`

Namespace	Meaning	Example
`pj:<id>:`	Project menu actions	`pj:42:verify`
`pl:<page>:<mode>`	Project list paging and filtering	`pl:1:archived`
`sr:<id>:`	Search result card actions	`sr:42:details`
`sp:<page>`	Search result paging	`sp:2`
`ad:`	Administrative actions	`ad:u:12345:revoke`

11.5 Free text steps

Several flows need a free text answer: a project name, a collection name, a metadata value, a search phrase, a new password. Rather than a conversation state machine, the application keeps at most one pending prompt per user, tagged with its kind, its parameters and its creation time.

The model has three useful properties. Any command or button press clears the pending prompt, so a stray message can never be captured as an answer to an abandoned step. Prompts expire after thirty minutes and the user is told the step expired rather than being silently ignored. And because the prompt is a single small record, the routing logic is one readable dispatch rather than a graph of states.

Passwords are handled with particular care: the message containing a password is deleted on a best effort basis as soon as it has been read, so the secret does not remain in the chat history.

11.6 Message construction

All outgoing messages use Telegram's HTML parse mode. Every interpolated value passes through an escaping helper, and link previews are disabled so that a Drive link does not expand into a large card. Designer mentions use the `tg://user` form, which works even for people who have no public username.

11.7 The live status message

A naive implementation posts a fresh progress message every time a scan runs. At a thirty minute interval across a handful of open projects, that buries the MG Group in near identical messages and trains people to ignore it.

Instead each project owns exactly one progress message in its group, identified by `status_message_id`, and the bot edits that message in place.

Trigger	Behaviour
Scheduled scan, private **Check progress**, and the verify, reopen, restore and revoke transitions	Edit the existing message silently. Nothing is created, so background work never adds to the chat
`/status`	Move the message to the bottom of the chat with current figures. The previous copy is deleted, or relabelled as outdated when Telegram no longer permits deleting it, which is 48 hours
Reminders, and the ready, archived and cancelled notices	Posted as separate messages, because their purpose is to notify people rather than to record state

Three properties follow from the design and are worth stating because each was a deliberate choice.

1. **The bot never creates the message by itself.** The first `/status` in a group creates it. A group that nobody has asked for status in therefore receives no progress traffic at all.
2. **An unchanged message is never rewritten.** `status_message_hash` holds a digest of the rendered text, and an edit that would produce the same text is skipped, which avoids both the API call and the "edited" marker appearing on a message that did not change.
3. **Losing the message is harmless.** If a group administrator deletes it, the next edit fails, the stored identifier is cleared and scans fall silent again until the next `/status` re-creates it. Relinking a project to a different group clears it for the same reason.

A scan whose report carried Drive errors never becomes the live message, so the last known good state stays on display. In that case `/status` replies with a separate warning instead.

11.8 Group identity and renaming

A Telegram group's name can change at any time, and a stale name in the project menu or the index sheet is a small but persistent annoyance. The stored name is kept current from three directions:

- A `new_chat_title` service message updates it immediately.
- The authorisation gate writes the name through on the way past, so ordinary use keeps it fresh at no extra cost. This covers `/status`, `/files` and `/remind`, the three group commands that pass through the gate. The group `/help` is ungated and refreshes nothing.
- A job every six hours, first run 30 seconds after start, asks Telegram for each authorised group's name. A rename made while the bot was stopped raises no service message, so without this job it would go unnoticed until somebody next ran a gated command in that group.

11.9 Chat migration

When a basic group is upgraded to a supergroup its chat identifier changes. This is handled twice, because the migration can be observed through either path. A service message handler re-keys the group record and every project that referenced the old identifier, and every outbound group send catches the migration error, re-keys, and retries the send. A live status message that is reposted during a migration is recorded against the new chat identifier, so the project does not end up pointing at a message in a chat that no longer exists. Authorisation therefore survives an upgrade without operator action.

12 Security

12.1 Attack surface

The process exposes no listening socket. All communication is outbound HTTPS to Telegram and Google, so there is nothing to port scan and no service to flood directly. The realistic attack surface is therefore the set of messages an attacker can cause the bot to receive, plus the secrets held on the host.

12.2 Authentication

Access is granted once per Telegram account by a shared password. On `/start` an unknown user is prompted; a correct password registers them as a Designer and the password is never requested again. The account identity is the Telegram user id, which the client supplies and the user cannot forge.

The password is stored only as a salted scrypt hash, using the standard library implementation with a work factor of 2^14, block size 8, parallelism 1, a 32 byte derived key and a 16 byte random salt. Verification uses a constant time comparison. The initial password is seeded once from the environment on first start and is rotated afterwards with `/setpassword`, which enforces a minimum length of eight characters.

A revoked account cannot re-register with the password. This is what makes revocation meaningful: removing access does not merely log someone out, it prevents them from coming back through the front door.

12.3 Authorisation

Capability	Super Admin	Team Lead	Designer	Lights
Search the archive and play previews	Yes	Yes	Yes	Yes
Open the Drive folder, read full details, list files	Yes	Yes	Yes	No
Be assigned responsibility for an asset	Yes	Yes	Yes	No
Create and manage projects, assign designers, verify	Yes	Yes	No	No
Create MG Group provisioning tokens	Yes	Yes	No	No
Project index sheet	Yes	Yes	No	No
Manage users, roles and revocation	Yes	No	No	No
Manage and revoke MG Groups	Yes	No	No	No
Change the access password	Yes	No	No	No

Authorisation is enforced by a single decorator applied to every handler, which checks the chat scope, the existence and active status of the actor, the minimum role and, in a group, the group's authorisation. The Super Admin role is pinned to the configured Telegram id: it is granted automatically to that account, it cannot be assigned to anyone else, that account cannot be revoked or demoted, and at every startup any other row holding the role is demoted to Team Lead.

The group rule is the conjunction required by the specification: an action in a group requires **both** an authorised user **and** an authorised MG Group. An unauthorised user in an authorised group is denied, and an

authorised user in an unauthorised group is denied.

12.4 The Lights role

Lights operators need to find a song and watch its preview during a service. They do not contribute assets and have no reason to reach the archive itself. The `LIGHTS` role expresses exactly that: it ranks alongside `DESIGNER`, so it clears the same private chat gate, but its surface is cut down to search and previews.

Surface	Behaviour for Lights
Search	Works normally. The result card carries a single **Preview** button
Open Archive, Details, Files	Not offered. Open Archive is a link button and is simply absent; the Details and Files callbacks are refused if replayed
`/files` in a group	Refused with an explanation
`/status` in a group	Allowed, because the group can already see that message
Assignment to a project	Impossible. Lights are filtered out of every assignment list, and the service layer rejects the attempt by name

Two points of design are worth drawing out.

The restriction is enforced at the handler, not merely by hiding buttons. A Lights user who replays a captured callback payload for **Details** or **Files** is refused, so no Drive link and no file name reaches them by that route. Hiding a button is presentation; refusing the callback is the control.

Exclusion from assignment is enforced in `services/projects.py` rather than only in the keyboard that lists candidates. `CONTRIBUTOR_ROLES` names the roles that can hold an assignment, `list_assignable_users` filters the pickers, and `toggle_assignment` raises with the person's name and role if something slips through. This matters because an assigned Lights user would be mentioned in reminders for work they cannot deliver.

The role is a plain string column, so introducing it required no database migration.

12.5 Group authorisation

1. A Team Lead requests a token in a private chat. The token has the form `MG-XXXX-XXXX`, drawn from a 32 character alphabet that omits the visually ambiguous characters, giving roughly 1.1 x 10^12 combinations. It is single use and expires after `PROVISIONING_TOKEN_TTL_HOURS`, by default 24 hours. Issuing a new token expires that person's earlier unused tokens.
2. The Team Lead creates the Telegram group and adds the bot.
3. If the person who added the bot holds exactly one valid unused token, the group is authorised automatically. Otherwise the bot asks for `/activate MG-XXXX-XXXX` to be sent in the group, which any active Team Lead or Super Admin may do.
4. A chat that is neither authorised nor activated within `UNAUTHORISED_GROUP_LEAVE_MINUTES` is left automatically by a job that runs every ten minutes. The bot also leaves any channel it is added to.
5. A group revoked deliberately by the Super Admin cannot be re-activated with a Team Lead's token; only the Super Admin can restore it. A group the bot was merely removed from can be re-activated normally. The distinction is recorded in the group row, so a deliberate revocation cannot be worked around.

12.6 Visibility of registrations

Access is granted by a shared password, so the Super Admin cannot see who used it unless the system tells them. Every successful registration therefore produces a direct message naming the person, their Telegram handle if they have one, their numeric identifier and the role they were given, with two buttons that promote them to Team Lead or open their user card directly.

The control this provides is practical rather than theoretical: an unexpected name arriving is the signal that the password has spread beyond the ministry, and the response, revoking that account and rotating the password, is two taps away from the notice itself. The Super Admin's own registration is not announced to them.

12.7 Abuse controls

Control	Threshold	Effect
Per account lockout	`LOGIN_MAX_FAILURES`, default 5	The account is locked for `LOGIN_LOCKOUT_MINUTES`, default 15, and the Super Admin is alerted by direct message
Global circuit breaker	`PASSWORD_BREAKER_FAILURES`, default 30, within `PASSWORD_BREAKER_WINDOW_MINUTES`, default 10	Registration is paused for `PASSWORD_BREAKER_PAUSE_MINUTES`, default 15, with a single alert. This bounds a distributed guessing attempt that spreads attempts across many accounts
Unknown sender throttle	One reply per `UNREGISTERED_REPLY_INTERVAL_SECONDS`, default 60	The bot cannot be used as a reply amplifier against a third party
Group status and listing cooldown	`STATUS_COOLDOWN_SECONDS`, default 60	Repeated `/status` reuses the last scan, and a repeated file listing reuses the last read, instead of issuing fresh Drive calls
Stale chat eviction	`UNAUTHORISED_GROUP_LEAVE_MINUTES`, default 60	The bot leaves chats it was added to without authorisation

12.8 Input handling

Vector	Control
Message text rendered back to users	HTML escaped through a single helper before interpolation
Database access	Parameterised throughout via the ORM; no string built SQL
Drive query strings	Backslashes and quotes escaped before interpolation
Drive file names used locally	Never used as path components; downloads use a fixed name inside a per job directory, and preview names are reduced to a safe character set
Subprocess invocation	Argument list, never a shell string
Button payloads	Parsed defensively; authorisation is re-evaluated server side for every press
Free text values	Length capped per field, year range validated, project names validated and rejected if they contain angle brackets

12.9 Secrets

The bot token, the Super Admin identifier, the initial password and the Google credentials live only in the environment file and the secrets directory. Both are excluded from version control, and both should be readable only by the service account that runs the process. Nothing secret is written to the log or sent to a user. The repository itself contains no credentials, so publishing the source discloses nothing: a person who clones it can only run their own instance, against their own bot token, their own Google project and their own database.

Secret	If disclosed
Telegram bot token	Regenerate it in BotFather and update the environment file
Service account key	Delete the key in the Cloud Console and issue a new one
OAuth token	Revoke it in the Google account and re-run the bootstrap tool
Access password	Rotate with `/setpassword`, then review `/users` and revoke any unexpected account

Because version control retains history, rotation is the only remedy for a credential that was ever committed; removing the file from the working tree is not sufficient.

12.10 Logging and audit

The application logs to the console and to a rotating file, five megabytes with three generations by default. Third party libraries are turned down to warning level so the log stays readable. Registrations, project creation, verification, revocation, group authorisation and password changes are logged with the acting Telegram id. The `validation_runs` table provides a durable, structured audit trail of what the system observed on Drive and when.

13 Concurrency, reliability and correctness

13.1 Concurrency rules

The application processes updates concurrently, so one person's Drive scan never blocks another person's button press. Concurrency at this level is cheap to get wrong, so the codebase follows four explicit rules.

1. **Commit before awaiting.** Every database mutation is committed before the next Telegram or Drive call. No write transaction is held across an `await`, so a second writer never spins on a locked database.
2. **Answer each callback query exactly once.** Telegram rejects a second answer to the same query, so each button path has exactly one acknowledgement.
3. **Serialise per project.** Non idempotent actions, namely creating the tree, verifying, reopening, revoking, restoring and scanning, run under a lock keyed by project identifier and refresh the ORM object after acquiring it. A double tap or an overlapping scheduled scan therefore cannot repeat a transition or provision a second folder tree.
4. **Keep sessions on the event loop.** Worker threads receive plain data structures, never ORM objects or sessions.

13.2 Failure behaviour

Failure	Behaviour
Google Drive unreachable during a scan	The affected leaves are marked as errored, nothing is stored, no transition, no reminder, and the interface shows a warning
Google credentials expired	Reported as a Drive error carrying the specific remediation for the configured authentication mode
Sheets API unavailable or not enabled	The index falls behind; the bot continues; the Super Admin is notified once; the nightly rebuild repairs it
Bot removed from an MG Group	The group is marked revoked; posts stop; the failure to post is logged rather than raised at the user
The live status message cannot be edited	Treated as gone: the stored identifier is cleared, scans fall silent for that project, and the next group status command creates a new message
A folder cannot be listed during a file listing	That folder alone is marked unreadable and the rest of the tree is still rendered; a failure at the project root is reported instead of a listing
Group upgraded to a supergroup	Identifiers are re-keyed and the send is retried automatically
`ffmpeg` missing or failing	The preview row is marked failed with the captured error; the project's lifecycle is unaffected; the creator is notified
Insufficient disk space	Detected before the download starts; the job fails with the required and available figures
Process restart mid transcode	Working directories are purged at startup and pending previews are re-planned by the next scan
Telegram network error	Logged as a warning and retried by the client library; the user sees no error

The governing idea is that the system degrades rather than misreports. Where the truth is unknown, the system says so and changes nothing.

14 Configuration reference

Configuration is read from the environment, optionally through a `.env` file, and validated at startup by a typed settings model. An invalid value stops the process with a precise message naming the variable.

Variable	Default	Purpose
`TELEGRAM_BOT_TOKEN`	none, required	Bot token issued by BotFather
`SUPER_ADMIN_TELEGRAM_ID`	none, required	Numeric Telegram id of the Super Admin account
`INITIAL_ACCESS_PASSWORD`	empty	Seeds the access password on first start only
`LOGIN_MAX_FAILURES`	5	Failed password attempts before an account is locked
`LOGIN_LOCKOUT_MINUTES`	15	Duration of an account lockout
`PROVISIONING_TOKEN_TTL_HOURS`	24	Validity of a group provisioning token
`UNAUTHORISED_GROUP_LEAVE_MINUTES`	60	Delay before leaving an unauthorised chat; 0 disables eviction
`UNREGISTERED_REPLY_INTERVAL_SECONDS`	60	Minimum interval between replies to an unknown sender
`PASSWORD_BREAKER_FAILURES`	30	Global wrong password budget
`PASSWORD_BREAKER_WINDOW_MINUTES`	10	Window over which that budget is measured
`PASSWORD_BREAKER_PAUSE_MINUTES`	15	Registration pause once the budget is exhausted
`STATUS_COOLDOWN_SECONDS`	60	Reuse window shared by the group status command and the file listing
`DATABASE_URL`	`sqlite:///data/mg_archive.sqlite3`	Database location; an in memory database is rejected
`WORK_DIR`	`data/work`	Scratch space for preview transcoding
`GOOGLE_AUTH_MODE`	`service_account`	One of `service_account`, `oauth` or `fake`
`GOOGLE_SERVICE_ACCOUNT_FILE`	`secrets/service-account.json`	Service account key file
`GOOGLE_OAUTH_CLIENT_SECRETS_FILE`	`secrets/oauth-client.json`	OAuth client definition
`GOOGLE_OAUTH_TOKEN_FILE`	`secrets/oauth-token.json`	OAuth token produced by the bootstrap tool
`DRIVE_ROOT_FOLDER_ID`	empty, required unless faked	Folder that contains every project folder
`FOLDER_NAME_*`	Specification defaults	Individual overrides for each folder name
`FFMPEG_PATH`	`ffmpeg`	Path to the encoder

Variable	Default	Purpose
PREVIEWS_ENABLED	true	Master switch for preview generation
PREVIEW_MAX_WIDTH	1280	Maximum preview width in pixels
PREVIEW_CRF	26	Encoder quality; lower is larger and better
SCAN_INTERVAL_MINUTES	30	Interval between scheduled scans
REMINDER_HOUR	10	Local hour of the daily reminder, validated to the range 0 to 23
REMINDER_MIN_GAP_HOURS	20	Minimum interval between two reminders for the same project
TIMEZONE	host zone	IANA zone used for scheduling and display
TRACKING_SHEET_ENABLED	true	Master switch for the project index
TRACKING_SHEET_ID	empty	Use an existing spreadsheet instead of creating one
TRACKING_SHEET_TITLE	MG Archive Index	Title used when the spreadsheet is created
SEARCH_PAGE_SIZE	5	Search results per page
LOG_LEVEL	INFO	Logging verbosity
LOG_FILE	data/bot.log	Rotating log file; empty logs to the console only

15 Deployment and operations

15.1 Requirements

Requirement	Detail
Runtime	Python 3.11 or newer, or Docker
Encoder	`ffmpeg` on the host, unless previews are disabled. The provided image includes it
Host	An always on machine: a Mac mini, a small virtual server, a network storage appliance or a container host
Disk	Sufficient free space in the working directory for the largest master plus 30 per cent, plus a 200 MB margin
Network	Outbound HTTPS to Telegram and Google. No inbound ports
Google	A Cloud project with the Drive API enabled, the Sheets API enabled if the index is used, and a credential with content manager rights on the archive root

15.2 Installation

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env          # then set the required variables
python -m mg_archive_bot
```

The container route builds an image that already contains `ffmpeg` and mounts `data` and `secrets` as volumes, with the database and working directory pointed inside the data volume.

```
docker compose up -d --build
```

15.3 Operator tools

Three tools run outside the bot process and are invoked with `python -m`.

Tool	Purpose
`mg_archive_bot.tools.drive_check`	Report which Google identity the bot uses, whether the archive root is visible, whether it lives in a Shared Drive and whether folders can be created there, naming the exact permission to request when something is missing
`mg_archive_bot.tools.google_oauth`	One time bootstrap of an OAuth token, for the authentication mode that uses a normal Google account
`mg_archive_bot.tools.reset_password`	Set the access password without going through Telegram, and clear every login lockout

The password reset tool exists because of a trap in the configuration. `INITIAL_ACCESS_PASSWORD` is read exactly once, when the database is first created; editing it in the environment file afterwards changes nothing,

which is not obvious and has surprised operators. It also covers the case where the Super Admin cannot reach Telegram, or where everyone is locked out at once.

```
python -m mg_archive_bot.tools.reset_password             # take the password from INITIAL_ACCESS_PASSWORD
python -m mg_archive_bot.tools.reset_password --prompt    # type it, twice, without it being echoed
```

It rewrites the stored hash and deletes every row in `login_attempts`, so outstanding lockouts end immediately. Registered users keep their access: the password governs new registrations only, which means the tool is not a way to remove anybody. Revocation in `/users` remains the only way to do that. The tool exits non zero and changes nothing when the two prompted entries differ, or when the environment variable it was told to use is empty.

15.4 Startup sequence

1. Settings are loaded and validated; a configuration error exits with status 2 and a precise message.
2. Logging is configured for the console and the rotating file.
3. The working directory is created and any leftovers from an interrupted run are removed.
4. The database is opened, tables are created if absent and missing columns are added.
5. The access password is seeded if none exists, and any account incorrectly holding the Super Admin role is demoted.
6. The Drive client is constructed, the archive root is fetched and listed, and the Sheets client is constructed if the index is enabled.
7. The Telegram application is assembled, handlers and jobs are registered, the preview worker starts, command menus are published, and long polling begins.

15.5 Scheduled jobs

Job	Schedule	Function
`scan`	Every `SCAN_INTERVAL_MINUTES`, first run after two minutes	Validate all open projects and queue previews
`reminders`	Daily at `REMINDER_HOUR`	Post outstanding items into each MG Group
`leave-stale-chats`	Every ten minutes, first run after five	Leave chats that were never authorised
`refresh-group-titles`	Every six hours, first run 30 seconds after start	Ask Telegram for each authorised group's current name, which catches renames made while the bot was stopped
`rebuild-sheet`	Daily at 03:30	Rewrite the project index from the database

15.6 Backup and recovery

The state that matters is the database file, the environment file and the secrets directory. Google Drive holds the asset content and is covered by Google's own retention, including a 30 day trash window for anything the bot removes.

A cold backup is a copy of `data/mg_archive.sqlite3` together with the write ahead log while the process is stopped. For a hot backup, use SQLite's own backup facility rather than copying the file in place. Recovery is

a restore of the database file followed by a restart; the next scan reconciles the recorded state with what is actually on Drive, and the nightly job repairs the index sheet.

15.7 Upgrade procedure

1. Stop the service.
2. Take a backup of the database file.
3. Update the source and dependencies.
4. Start the service and watch the log. New columns are added automatically; a change that cannot be applied safely stops the process with an explicit message rather than continuing on a wrong schema.

15.8 Runbook

Symptom	Likely cause	Action
`Configuration problem: DRIVE_ROOT_FOLDER_ID is required`	The variable is unset	Set it, or use the fake mode for an evaluation run
Drive error with status 404 on project creation	The root folder is not shared with the bot's identity	Run the Drive diagnostic tool and grant content manager rights
Progress shows that Drive could not be checked	Outage, expired credential or a deleted folder	Inspect the log; no state changes until a scan succeeds
`ffmpeg not found` in a preview failure	The encoder is absent or not on the path	Install it or set `FFMPEG_PATH`
Group commands are refused as not authorised	The group was never activated, or was revoked	Issue a token with `/creategroup` and send `/activate` in the group
Reminders arrive at the wrong hour	The timezone is unset or wrong	Set `TIMEZONE` to the correct IANA zone and restart
The index sheet stops updating	The Sheets API is disabled, or permissions changed	Enable the API or repair access, then run `/sheet rebuild`
A preview never appears	The job failed and is not retried automatically	Open the project menu and request generation, which forces a retry
Nobody knows the access password, or everyone is locked out	The password was never recorded, or a guessing attempt locked the accounts	Run `python -m mg_archive_bot.tools.reset_password --prompt` on the host. Editing `INITIAL_ACCESS_PASSWORD` has no effect on an existing database
The progress message in a group stopped updating	It was deleted in the group, so the bot forgot it	Send `/status` in that group; the message is created again and scans resume editing it
A group shows an old name	The group was renamed while the bot was stopped	It corrects itself within six hours, or immediately on the next command sent in that group

The first diagnostic step for any unexplained behaviour is the log:

```
tail -n 50 data/bot.log
```

15.9 Performance and capacity

A scan issues one listing per required leaf plus one per sub folder it descends into, which is a small number of requests per project. With the default half hourly interval, a portfolio of a few hundred open projects is comfortably within Drive's quotas. Transcoding is the only expensive activity and it is serialised deliberately; a long master occupies the worker for the duration and other work continues unaffected on the event loop.

The known scaling limits and the points at which they would need attention are recorded in section 17.

16 Quality assurance

16.1 Approach

Testing concentrates on the domain logic and on the handler flows, both of which are exercised against test doubles rather than live services. Two doubles do the heavy lifting: an in memory Drive that implements the same protocol as the Google client including trash semantics, and a fake Telegram application that captures the messages and keyboards a handler produces. A stand-in encoder is used so that the transcoding path is exercised without invoking a real one. Every test runs against a temporary SQLite file.

16.2 Suite composition

Module	Tests	Focus
`test_core_services.py`	23	Users, roles, revocation, lockout, tokens, groups, group renaming, project creation and provisioning
`test_validation_search_previews.py`	8	Leaf satisfaction, error semantics, state transitions, ranking order, preview planning and transcoding
`test_jobs.py`	2	Scheduled scan and reminder behaviour
`test_handlers.py`	39	End to end command and button flows through the fake Telegram application, including the live status message, two chat routing and the Lights restrictions
Total	**72**	

16.3 Running the suite

```
pip install -r requirements-dev.txt
python -m pytest -q
```

16.4 Emphasis and gaps

The suite deliberately concentrates on behaviour that is expensive to verify manually and dangerous to get wrong: the ranking order, the rule that an unlistable folder is not an empty folder, the idempotency of the creation and verification paths, and the role and group authorisation matrix.

Two areas are covered only indirectly. The real Google clients are exercised by the startup diagnostic rather than by unit tests, because meaningful coverage would require live credentials. Long running transcodes are represented by a stand-in encoder, so encoder specific failures surface in operation rather than in the suite.

17 Constraints, risks and roadmap

17.1 Known constraints

Constraint	Consequence	Mitigation
Single process design	Throughput is bounded by one event loop and one transcoding worker	Adequate at the current scale; the database URL can be repointed at PostgreSQL if a second instance becomes necessary
Search executes in the application	Cost grows linearly with the number of projects	Negligible for thousands of projects; an indexed approach becomes worthwhile in the tens of thousands
SQLite	Single writer	Every handler commits before awaiting, so writes are brief; write ahead logging keeps readers unblocked
Previews are not retried automatically	A failed preview needs a manual request	Deliberate: an automatic retry would repeatedly download a multi gigabyte master
Drive is polled, not subscribed	Changes are noticed within the scan interval	Team Leads can force an immediate check from the project menu
A file listing reads six levels below the project folder	A designer who nests sub-folders more deeply than that sees the deeper levels reported as not shown	The depth covers the archive tree plus three levels of a designer's own structure, which is well beyond observed practice, and the omission is stated rather than silent

17.2 Risk register

Risk	Impact	Likelihood	Response
Google credential expires or is revoked	Scans and previews fail until repaired	Medium	Errors carry the specific remediation; scans change nothing while Drive is unreachable
Shared access password is leaked	An outsider registers	Low	Rotate with `/setpassword` and revoke unexpected accounts; a revoked account cannot re-register
Host disk fills during transcoding	Preview jobs fail	Medium	Space is checked before download; failures name the required and available amounts
Bot token disclosed	Full control of the bot identity	Low	Regenerate in BotFather; no user data is held by the token itself
Archive root moved or deleted on Drive	Provisioning and scans fail	Low	Startup verification fails fast with a clear message; Drive's own trash allows recovery
Sheets API quota or outage	Index falls behind	Low	Isolated from the main flow, reported once, repaired nightly

17.3 Candidate future work

1. PNG previews for Contin Lyrics, mirroring the MP4 pipeline for still deliveries.
2. Per designer digests, so an individual can see everything outstanding across projects in one message.
3. An optional PostgreSQL deployment for multi instance operation.
4. Full text indexing of names, tags and descriptions once the archive reaches a scale where the in application scorer becomes measurable.
5. Retention reporting over the `validation_runs` history, for example average time from request to completion by designer or by ministry.

Appendix A: Folder key reference

Key	Folder path	Validated	Preview source
working_file	Working File	No	No
fonts	Working File / Fonts	Always	No
ae	Working File / AE	Always	No
psd	Working File / PSD	When declared	No
final_render	Final Render	No	No
timeline	Final Render / Timeline	No	No
timeline_prores	Final Render / Timeline / ProRes 4444	When declared	Yes
timeline_hap	Final Render / Timeline / Hap/Hap Alpha	When declared	No
contin_videos	Final Render / Contin Videos	No	No
contin_prores	Final Render / Contin Videos / ProRes 4444	When declared	Yes
contin_hap	Final Render / Contin Videos / Hap/Hap Alpha	When declared	No
contin_lyrics	Final Render / Contin Lyrics	No	No
lyrics_png	Final Render / Contin Lyrics / PNG	When declared	No
previews	_Previews	No	Destination

Appendix B: Glossary of status labels

Stored value	Displayed as	Meaning for the reader
`DRAFT`	Draft	The creation wizard has not been confirmed
`ACTIVE`	Active	Open for uploads, nothing reported missing yet
`INCOMPLETE`	Incomplete	At least one declared asset has not been uploaded
`READY_FOR_VERIFICATION`	Ready for verification	Everything declared is present; a Team Lead must sign it off
`ARCHIVED`	Archived	Verified and closed
`CANCELLED`	Cancelled	Revoked; the Drive folder is in the trash and can be restored